CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Objective 12: Iterative Refinement

> **Exam objective:** Structure iterative refinement workflows by providing concrete input-output examples, targeted feedback on specific failures, and batched issue descriptions for consolidated evaluation.

Q1.

A team asks Claude Code to normalize API error payloads, but the first result changes field names inconsistently. What is the most effective next prompt?

A. "Try again, but better." B. "You failed to understand the assignment." C. "For input `{code:401,message:'bad token'}` the expected output is `{error:{type:'auth',message:'bad token'}}` ; keep existing names unless mapped explicitly." D. "Rewrite everything from scratch using your preferred schema."

Answer: C **Explanation:** Iterative refinement works best when feedback is concrete and example-driven. Providing a specific failing input-output pair and a targeted rule helps the model correct the exact defect. Vague criticism and complete restarts provide less actionable guidance.

Q2.

Claude Code generated tests that mock every function and assert only that methods were called. The team wants behavioral tests instead. Which feedback is most aligned with iterative refinement best practice?

A. "The tests are weak." B. "Write more sophisticated tests." C. "Replace call-count assertions with tests that verify externally visible behavior; for example, when input is X, the returned response should contain Y even if internals change." D. "Use a different language."

Answer: C **Explanation:** The best feedback names the specific failure mode and describes the desired behavior using an example. That gives the model a measurable correction target. Generic statements about quality are too ambiguous to drive reliable improvement.

Q3.

A developer notices five related formatting problems and three logic mistakes in an AI-generated patch. They want a single follow-up pass. How should they structure the refinement request?

A. Submit eight separate one-line prompts in random order B. Batch the issues into a consolidated list grouped by type, with enough detail for the model to address them together C. Ask the model to "inspect everything again" D. Discard the patch and start with an unrelated prompt

Answer: B **Explanation:** Batched issue descriptions enable the model to evaluate related problems in one pass and avoid oscillation between narrow prompts. Grouping issues by type improves coherence and reduces rework. Vague reinspection requests are less efficient and less precise.

Q4.

An architecture team is refining a code-generation workflow for SQL migrations. Which prompt addition most improves reproducibility across iterations?

A. "Use good judgment." B. "See prior output." C. A concrete before-and-after example showing an input schema change and the exact migration pattern expected D. "Be consistent with best practices."

Answer: C **Explanation:** Concrete examples anchor the model to the intended transformation, especially for structured outputs like migrations. General exhortations to use judgment or best practices are open to interpretation and produce less consistent iterations. Explicit examples make the target clearer.

Q5.

After an initial run, Claude Code misclassified two edge cases but handled the common path correctly. What is the best refinement strategy?

A. Rewrite the entire specification from scratch B. Provide targeted feedback focused on the misclassified edge cases and preserve the parts that already worked C. Ignore the errors because most outputs were correct D. Add unrelated style instructions to increase rigor

Answer: B **Explanation:** Targeted feedback minimizes churn by correcting only the identified failures while reinforcing successful behavior. Restarting the whole specification can introduce regressions, and ignoring edge cases leaves known defects unresolved. Unrelated style guidance distracts from the real issue.

Q6.

A prompt engineer wants Claude Code to transform legacy config files into a new YAML format. The first attempt omits one nested section. Which refinement is strongest?

A. "You forgot part of the config." B. "Include nested data." C. "In this example, the legacy `retry.max=3` and `retry.backoff=exp` settings must appear under `retry:` in YAML as shown below..." D. "Read the docs more carefully."

Answer: C **Explanation:** The strongest refinement ties the missing behavior to a specific example and expected output structure. That reduces ambiguity and helps the model generalize correctly. Vague statements about omission provide too little signal.

Q7.

Why is batching multiple known defects into one structured follow-up often preferable to prompting one issue at a time?

A. It guarantees the model will never introduce new bugs B. It lets the model evaluate interactions among related issues and produce a more consolidated revision C. It reduces token usage to zero D. It makes examples unnecessary

Answer: B **Explanation:** Consolidated evaluation is valuable because some issues interact, and fixing them together can reduce churn and contradictory edits. It does not eliminate all regressions or examples, nor does it make token usage disappear. The benefit is coordination and efficiency.

Q8.

A generated parser handles valid inputs correctly but crashes on blank lines. Which follow-up instruction best demonstrates targeted feedback?

A. "Handle errors better." B. "Blank lines currently throw an index error; update the parser so blank or whitespace-only lines are skipped and return the same output as if they were absent." C. "The code quality is disappointing." D. "Refactor for elegance."

Answer: B **Explanation:** This instruction identifies the exact current failure, the desired behavior, and the expected semantic outcome. That specificity makes the refinement actionable. General requests for better error handling or elegance do not clearly identify the defect.

Q9.

A team wants Claude Code to generate reviewer comments for failed policy checks. The first output is close, but the phrasing is not compliant with internal standards. What is the best refinement input?

A. One example of a noncompliant comment and the exact compliant rewrite expected B. A request to "sound more professional" C. No feedback, because tone cannot be refined D. A demand for shorter answers only

Answer: A **Explanation:** Example-based refinements are especially effective for style and phrasing constraints. Showing the model the delta between noncompliant and compliant wording provides a concrete target. Vague style descriptors are harder to operationalize consistently.

Q10.

Which scenario best justifies including multiple input-output examples in a refinement workflow?

A. When the task has several important variants that differ in structure or edge-case handling B. When the team wants the shortest possible prompt regardless of ambiguity C. When the model already behaves perfectly D. When no one knows what correct output should look like

Answer: A **Explanation:** Multiple examples help the model infer patterns across meaningful variants, especially when structure or edge cases differ. If correctness is unknown, examples cannot be defined well; if behavior is already perfect, refinement is unnecessary. Prompt brevity should not override clarity.

Q11.

An SRE team is refining an incident-summary generator. The first summary mixes timeline facts with speculation. What is the best follow-up?

A. "Avoid mistakes." B. "Separate verified events from hypotheses; for example, place observed log timestamps in a Timeline section and any inferred causes in a Hypotheses section." C. "Be more careful with incidents." D. "Use a different output length."

Answer: B **Explanation:** This feedback isolates the exact failure and supplies a structural correction with a concrete example. It tells the model what distinction matters and how to express it. Generic admonitions provide little guidance on what to change.

Q12.

During iterative refinement, why is it useful to preserve successful constraints from earlier prompts instead of rewriting everything each time?

A. Because models ignore new instructions if they differ from earlier ones B. Because retaining working constraints reduces regression risk and focuses the next pass on unresolved failures C. Because shorter prompts are always more accurate D. Because refinement should never mention prior outputs

Answer: B **Explanation:** Iterative refinement is most effective when it builds on what already works. Preserving successful constraints narrows the delta for the next pass and reduces accidental regressions. Rewriting everything can blur priorities and destabilize good behavior.

Q13.

A team evaluates AI-generated SQL and sees three recurring issues: missing transaction wrappers, inconsistent constraint names, and omission of rollback statements. Which refinement prompt is best?

A. "Fix the SQL problems." B. "Address these three issues together: wrap each migration in a transaction, use the `ck_<table>_<column>` naming convention, and include explicit rollback SQL matching each forward change." C. "Make it production ready." D. "Rethink your approach."

Answer: B **Explanation:** The best prompt batches the known issues and states each desired correction concretely. That enables one consolidated evaluation pass and reduces ambiguity. Generic quality requests do not tell the model what production-ready means here.

Q14.

Claude Code produced a nearly correct config converter, but one sample containing duplicate keys should raise an error instead of silently overwriting data. Which refinement is strongest?

A. "Do more validation." B. "When the same key appears twice in the source, emit an error instead of keeping the last value; for example, input X should return error Y." C. "This is dangerous." D. "Use stricter parsing somehow."

Answer: B **Explanation:** This refinement pinpoints the failing condition, desired behavior, and an example outcome. That makes the correction testable and concrete. The other options identify concern but not the exact operational change needed.

Q15.

What is the main benefit of giving Claude Code targeted feedback on specific failures rather than only an overall score such as "7/10"?

A. Targeted feedback identifies actionable corrections the model can apply in the next iteration B. Numerical scores automatically retrain the model C. Overall scores reduce the need for examples in all cases D. Scores guarantee deterministic outputs

Answer: A **Explanation:** Specific, actionable feedback tells the model what failed and how to improve, which is the core of iterative refinement. Numeric scores alone are poor guidance because they do not reveal the nature of the defect. They neither retrain the model nor guarantee determinism.